

Отчёт по лабораторной работе №6

Выполнила студентка НКАбд-02-25

Валерия Сергеевна Григорьева

Содержание

1	Цель работы	3
2	Порядок выполнения лабораторной работы	4
2.1	Символьные и численные данные в NASM	4
2.2	Выполнение арифметических операций в NASM	14
3	Задание для самостоятельной работы	24
4	Вывод	28

1 Цель работы

Цель данной работы - освоение арифметических инструкций языка ассемблера NASM.

2 Порядок выполнения лабораторной работы

2.1 Символьные и численные данные в NASM

Для начала работы я создала каталог для лабораторной работы №6, перешла в него и создала там файл lab6-1.asm (Рисунок 2.1).

```
vsgrigoreva@fedora:~$ mkdir ~/work/arch--pc/lab06
vsgrigoreva@fedora:~$ cd ~/work/arch--pc/lab06
vsgrigoreva@fedora:~/work/arch--pc/lab06$ touch lab6-1.asm
```

Рисунок 2.1: Создание каталога и файла для работы

Затем я ввела в файл lab6-1.asm текст программы из листинга 1 (Рисунок 2.2).

```
%include 'in_out.asm'
```

```
SECTION .bss
```

```
buf1: RESB 80
```

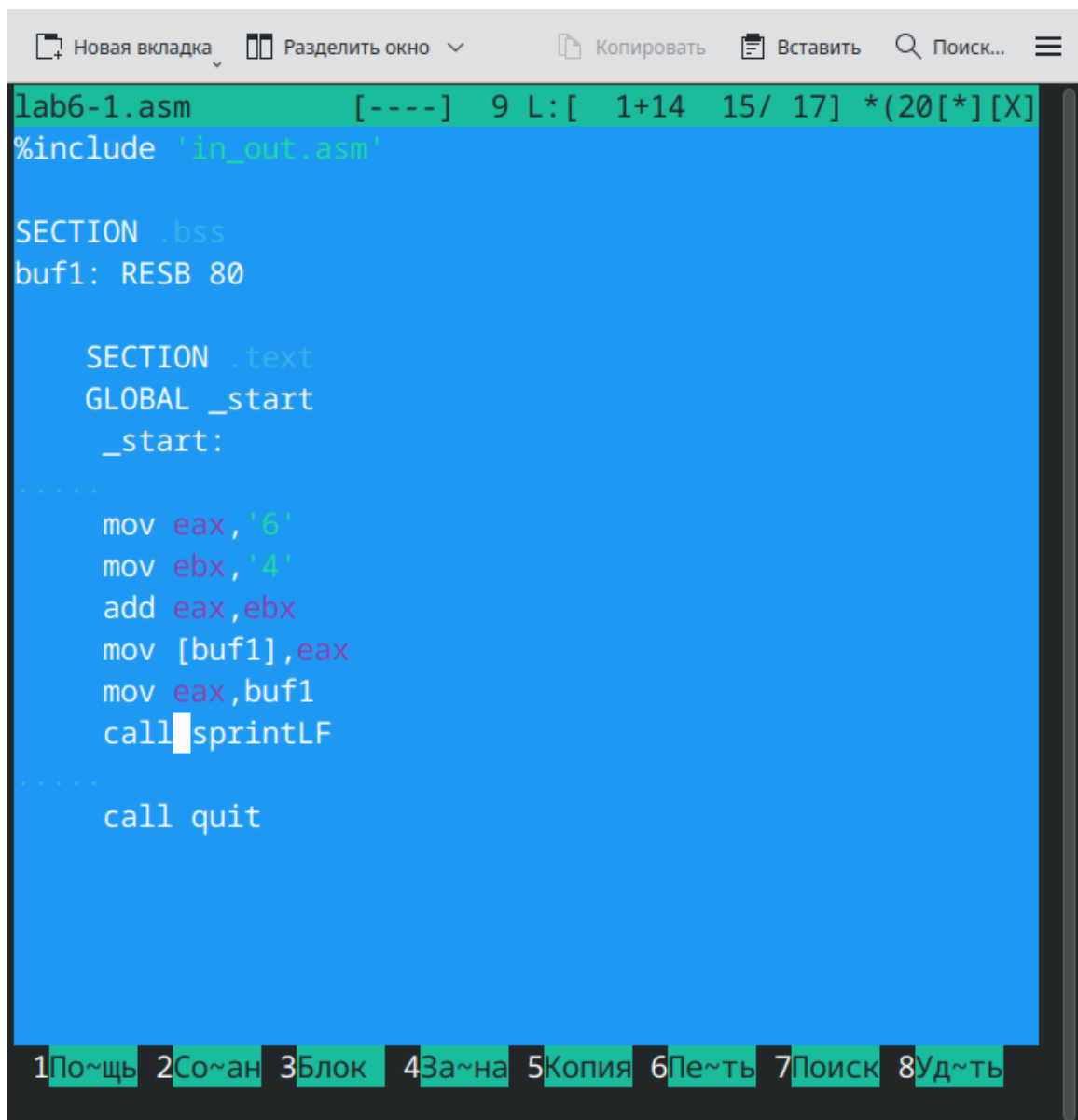
```
SECTION .text
```

```
GLOBAL _start
```

```
_start:
```

```
mov eax, '6'
mov ebx, '4'
add eax, ebx
mov [buf1], eax
mov eax, buf1
call sprintf

call quit
```



```
lab6-1.asm [----] 9 L:[ 1+14 15/ 17] *(20[*])[X]
#include 'in_out.asm'

SECTION .bss
buf1: RESB 80

SECTION .text
GLOBAL _start
_start:
.....
mov eax,'6'
mov ebx,'4'
add eax,ebx
mov [buf1],eax
mov eax,buf1
call sprintLF
.....
call quit
```

Рисунок 2.2: Текст программы из файла lab6-1.asm

В данной программе в регистр `eax` записывается символ 6 (`mov eax,,'6'`), в регистр `ebx` символ 4 (`mov ebx,,'4'`). Далее к значению в регистре `eax` прибавляется значение регистра `ebx` (`add eax,ebx`, результат сложения запишется в регистр `eax`). Далее выводится результат. Так как для работы функции `sprintLF` в регистр `eax` должен быть записан адрес, необходимо использовать дополнительную переменную. Для этого запишем значение регистра `eax` в переменную `buf1` (`mov [buf1],eax`), а затем запишем

адрес переменной buf1 в регистр eax (mov eax,buf1) и вызовем функцию sprintLF.

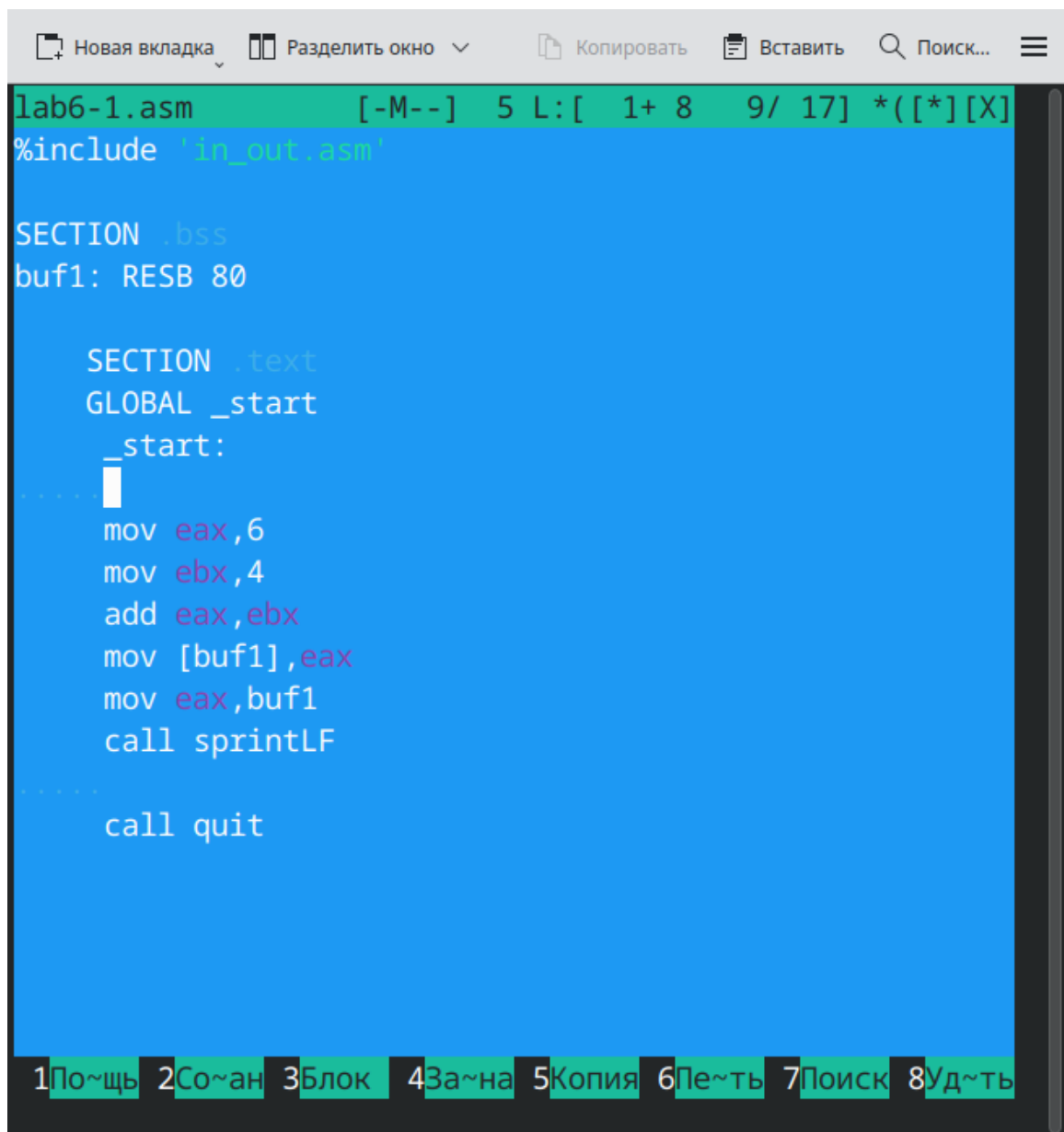
Затем я создала исполняемый файл и запустила его (Рисунок 2.3). Для корректной работы программы скопировала файл in_out.asm в каталог с файлом lab6-1.asm.

```
vsgrigoreva@fedora:~/work/arch--pc/lab06$ nasm -f elf lab6-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ./lab6-1
j
vsgrigoreva@fedora:~/work/arch--pc/lab06$ █
```

Рисунок 2.3: Создание исполняемого файла и запуск программы

Программа вывела не число 10, а символ j, потому что код символа 6 равен 00110110 в двоичном представлении (или 54 в десятичном представлении), а код символа 4 – 00110100 (52). Команда add eax,ebx запишет в регистр eax сумму кодов – 01101010 (106), что в свою очередь является кодом символа j в таблицу ASCII.

Далее изменила текст программы. Строки mov eax,,,"6" и mov ebx,,,"4" заменила на mov eax,6 mov ebx,4 (Рисунок 2.4).



```
lab6-1.asm [-M--] 5 L:[ 1+ 8 9/ 17] *([*])[X]
#include 'in_out.asm'

SECTION .bss
buf1: RESB 80

SECTION .text
GLOBAL _start
_start:
.....
    mov eax,6
    mov ebx,4
    add eax,ebx
    mov [buf1],eax
    mov eax,buf1
    call printf
.....
    call quit
```

1Помощь 2Сохранить 3Блок 4Замена 5Копия 6Печать 7Поиск 8Удалить

Рисунок 2.4: Текст измененной программы

Затем создала исполняемый файл и запустила его (Рисунок 2.5).


```

vsgrigoreva@fedora:~/work/arch--pc/lab06$ nasm -f elf lab6-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ./lab6-1

vsgrigoreva@fedora:~/work/arch--pc/lab06$ █

```

Рисунок 2.5: Создание исполняемого файла и запуск программы

Как и в предыдущем случае программа не вывела число 10, она должна была вывести символ с кодом 10 (черный квадрат с белым кругом внутри), но этот символ не отобразился на экране.

Далее необходимо было создать файл lab6-2.asm в каталоге ~/work/arch-pc/lab06 (Рисунок 2.6).

```

vsgrigoreva@fedora:~/work/arch--pc/lab06$ touch ~/work/
arch--pc/lab06/lab6-2.asm

```

Рисунок 2.6: Создание файла

Затем я ввела в него текст программы из листинга 2 (Рисунок 2.7).

```

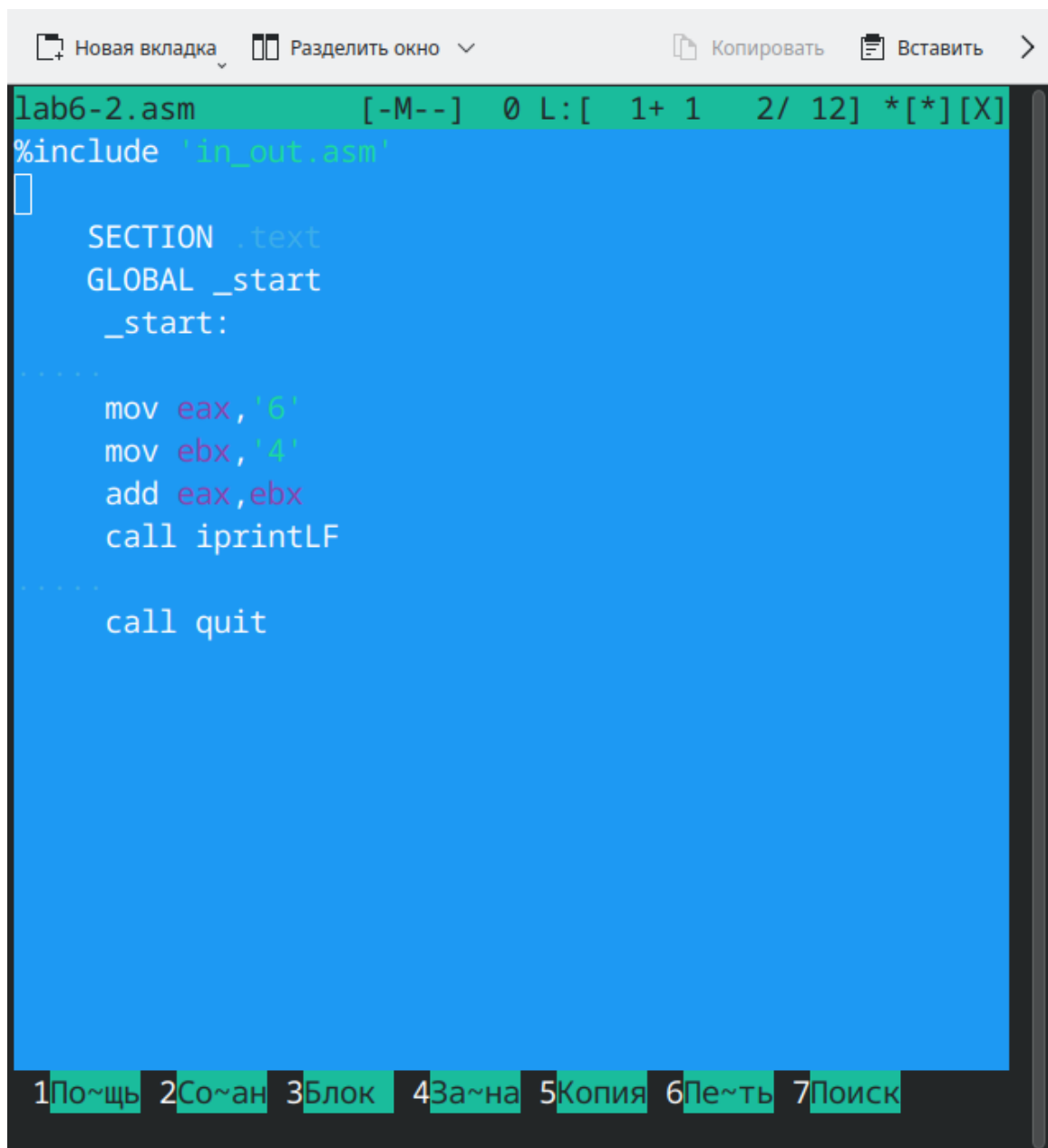
#include 'in_out.asm'

SECTION .text
GLOBAL _start
_start:

    mov eax, '6'
    mov ebx, '4'
    add eax, ebx
    call iprintLF

    call quit

```



```
lab6-2.asm [-M--] 0 L:[ 1+ 1 2/ 12] *[*][X]
#include 'in_out.asm'

SECTION .text
GLOBAL _start
_start:
.....
mov eax,'6'
mov ebx,'4'
add eax,ebx
call iprintLF
.....
call quit
```

1Помощь 2Сохранить 3Блок 4Замена 5Копия 6Печать 7Поиск

Рисунок 2.7: Текст программы из файла lab6-2.asm

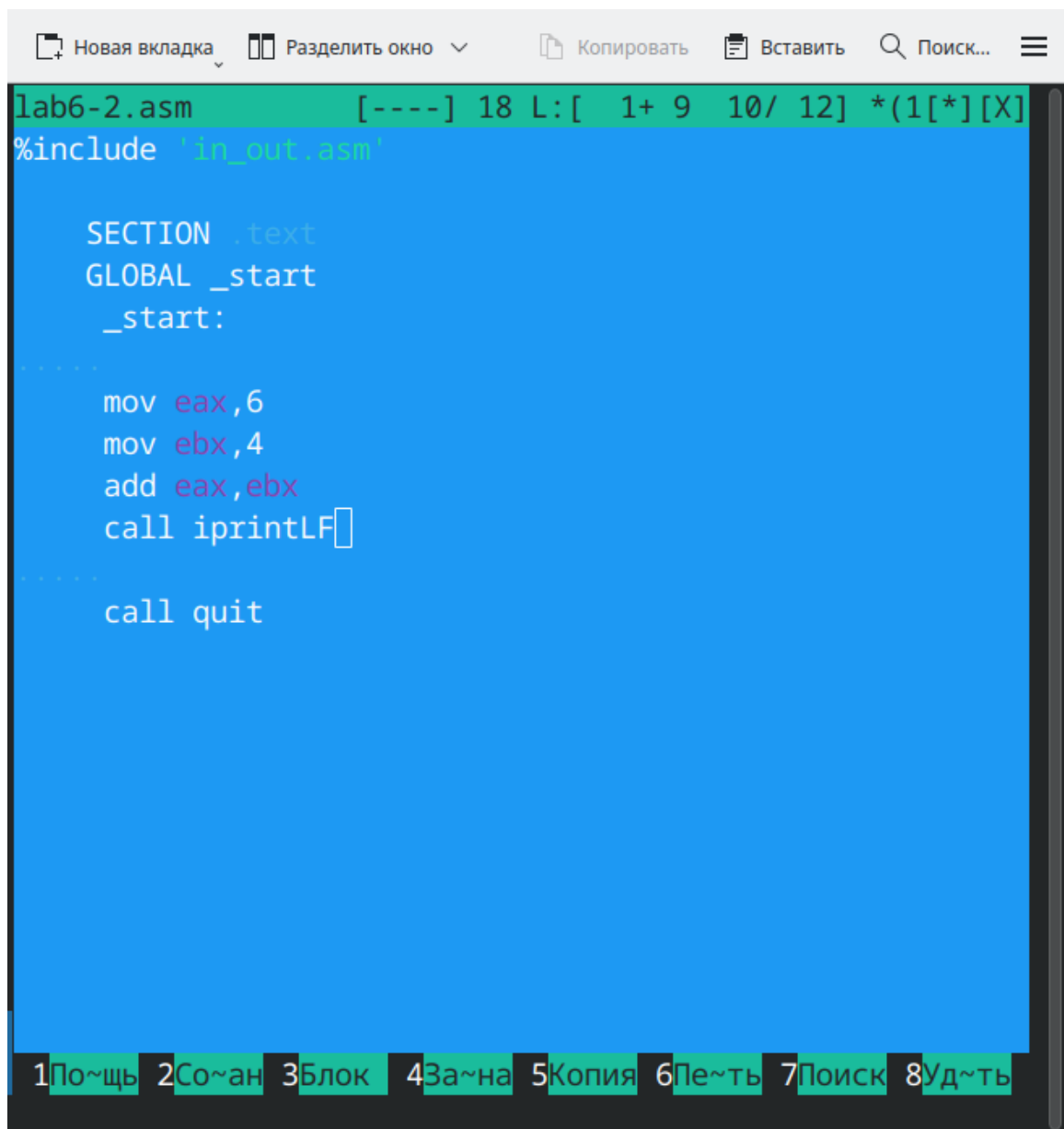
Затем я создала исполняемый файл и запустила его (Рисунок 2.8).

```
vsgrigoreva@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
vsgrigoreva@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
vsgrigoreva@fedora:~/work/arch-pc/lab06$ ./lab6-2
106
vsgrigoreva@fedora:~/work/arch-pc/lab06$
```

Рисунок 2.8: Создание исполняемого файла и запуск программы

В результате работы программы на экран вывелось число 106. В данном случае, как и в первом, команда `add` складывает коды символов „6“ и „4“ ($54+52=106$). Однако, в отличие от программы из листинга 1, функция `iprintLF` позволяет вывести число, а не символ, кодом которого является это число.

Затем в программе `lab6-2.asm` я заменила строки `mov eax,,6` и `mov ebx,,4` на строки `mov eax,6` и `mov ebx,4` (Рисунок 2.9).



```
lab6-2.asm      [----] 18 L:[ 1+ 9 10/ 12] *(1[*][X]
#include 'in_out.asm'

SECTION .text
GLOBAL _start
_start:
.....
mov eax,6
mov ebx,4
add eax,ebx
call iprintLF
.....
call quit
```

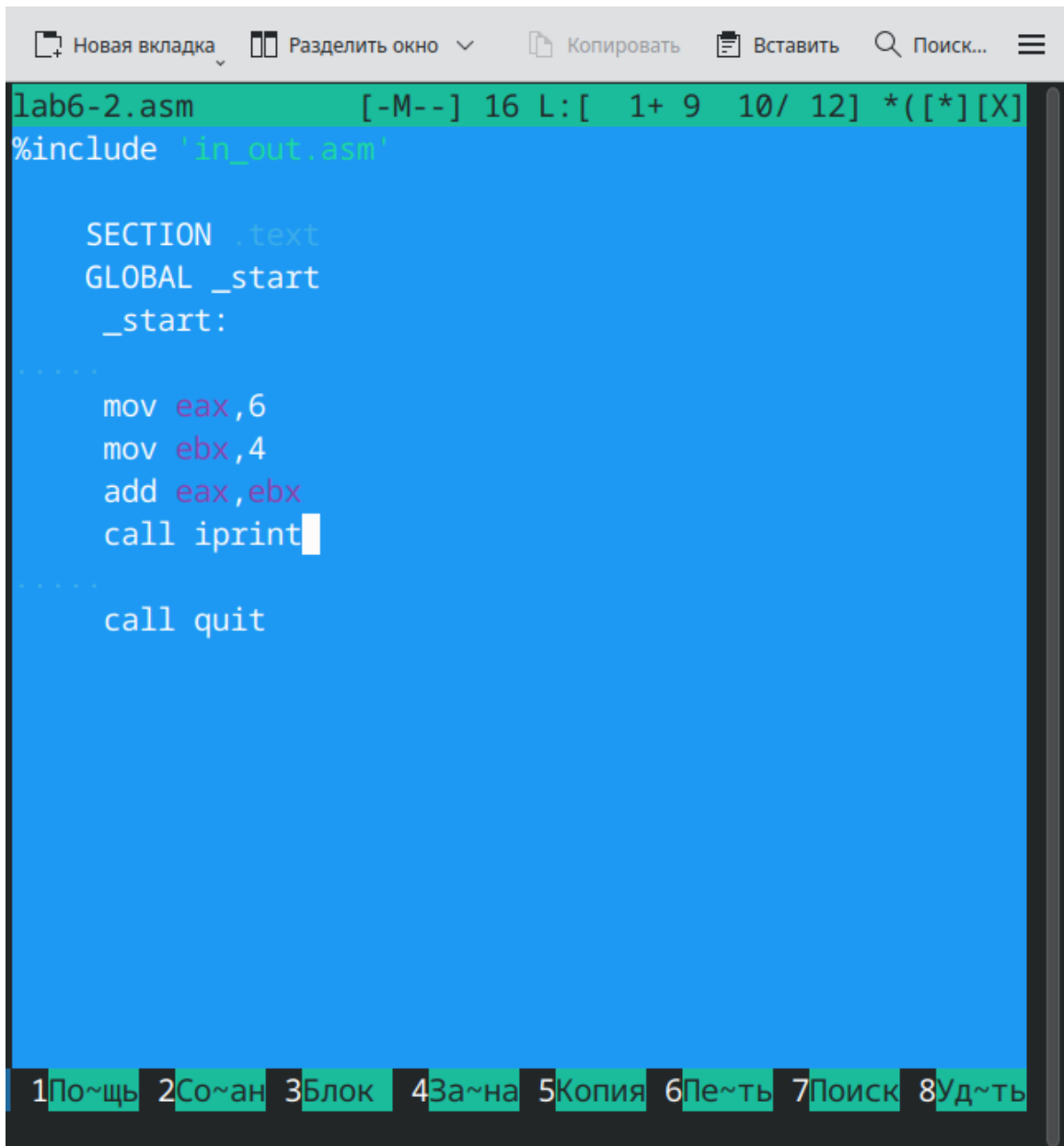
Рисунок 2.9: Измененный текст программы из файла lab6-2.asm

Далее я создала исполняемый файл и запустила его (Рисунок 2.10). Программа вывела число 10.

```
vsgrigoreva@fedora:~/work/arch--pc/lab06$ nasm -f elf lab6-2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ./lab6-2 10
vsgrigoreva@fedora:~/work/arch--pc/lab06$
```

Рисунок 2.10: Запуск программы из файла lab6-2.asm

Затем я заменила функцию `iprintLF` на `iprint` (Рисунок 2.11).



```
lab6-2.asm [-M--] 16 L:[ 1+ 9 10/ 12] *([*][X]
#include 'in_out.asm'

SECTION .text
GLOBAL _start
_start:
.....
mov eax,6
mov ebx,4
add eax,ebx
call iprint
.....
call quit
```

Рисунок 2.11: Измененный текст программы из файла lab6-2.asm

Далее я создала исполняемый файл и запустила его (Рисунок 2.12). Программа вывела число 10, но в начале следующей строки.

```
vsgrigoreva@fedora:~/work/arch--pc/lab06$ nasm -f elf lab6-2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ./lab6-2
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ./lab6-2
10vsgrigoreva@fedora:~/work/arch--pc/lab06$
```

Рисунок 2.12: Запуск программы из файла lab6-2.asm

2.2 Выполнение арифметических операций в NASM

Сначала я создала файл lab6-3.asm в каталоге ~/work/arch-pc/lab06 (Рисунок 2.13).

```
vsgrigoreva@fedora:~/work/arch--pc/lab06$ touch ~/work/arch--pc/lab06/lab6-3.asm
```

Рисунок 2.13: Создание файла lab6-3.asm

Далее вставила в него текст программы из листинга 3 (Рисунок 2.14).

```
;-----
; Программа вычисления выражения
;-----

%include 'in_out.asm' ; подключение внешнего файла

SECTION .data

div: DB 'Результат: ',0
rem: DB 'Остаток от деления: ',0

SECTION .text
GLOBAL _start
```

_start:

; ---- Вычисление выражения

mov eax,5 ; EAX=5

mov ebx,2 ; EBX=2

mul ebx ; EAX=EAX*EBX

add eax,3 ; EAX=EAX+3

xor edx,edx ; обнуляем EDX для корректной работы div

mov ebx,3 ; EBX=3

div ebx ; EAX=EAX/3, EDX=остаток от деления

mov edi,eax ; запись результата вычисления в 'edi'

; ---- Вывод результата на экран

mov eax,div ; вызов подпрограммы печати

call sprint ; сообщения 'Результат: '

mov eax,edi ; вызов подпрограммы печати значения

call iprintLF ; из 'edi' в виде символов

mov eax,rem ; вызов подпрограммы печати

call sprint ; сообщения 'Остаток от деления: '

mov eax,edx ; вызов подпрограммы печати значения

call iprintLF ; из 'edx' (остаток) в виде символов

call quit ; вызов подпрограммы завершения

```
lab6-3.asm [-M--] 22 L: [ 3+13 16/ 38] *(443 /1503b) 00[*][X]
;-----
#include 'in_out.asm' ; подключение внешнего файла

SECTION .data
----
div: DB 'Результат: ',0
rem: DB 'Остаток от деления: ',0

SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения
mov eax,5 ; EAX=5
mov ebx,2 ; EBX=2
mul ebx ; EAX=EAX*EBX
add eax,3 ; EAX=EAX+3
xor edx,edx ; обнуляем EDX для корректной работы div
mov ebx,3 ; EBX=3
div ebx ; EAX=EAX/3, EDX=остаток от деления

mov edi,eax ; запись результата вычисления в 'edi'
```

1По~щъ 2Со~ан 3Блок 4За~на 5Копия 6Пе~ть 7Поиск 8Уд~ть 9Ме~МС10Выход

Рисунок 2.14: Текст программы в файле lab6-3.asm

Далее я создала исполняемый файл и запустила его (Рисунок 2.15).

```
vsgrigoreva@fedora:~/work/arch--pc/lab06$ nasm -f elf lab6-3.asm
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ./lab6-3
Результат: 4
Остаток от деления: 1
vsgrigoreva@fedora:~/work/arch--pc/lab06$
```

Рисунок 2.15: Запуск программы из файла lab6-3.asm

Далее я изменила текст программы для вычисления выражения $f(x) = (4*6+2)/5$ (Рисунок 2.16). Измененный текст приведен в листинге 4.


```

;-----
; Программа вычисления выражения  $f(x) = (4 * 6 + 2) / 5$ 
;-----

%include 'in_out.asm' ; подключение внешнего файла

SECTION .data
div: DB 'Результат: ',0
rem: DB 'Остаток от деления: ',0

SECTION .text
GLOBAL _start

_start:
    ; ---- Вычисление выражения ----
    mov eax,4      ; EAX = 4
    mov ebx,6      ; EBX = 6
    mul ebx        ; EAX = 4 * 6 = 24

    add eax,2      ; EAX = 24 + 2 = 26

    xor edx,edx    ; Обнуляем EDX перед делением
    mov ebx,5      ; EBX = 5
    div ebx        ; EAX = 26 / 5 = 5, EDX = 1 (остаток)

    mov edi,eax    ; Сохраняем результат в EDI

    ; ---- Вывод результата ----

```

```
mov eax,div
call sprint      ; Печать "Результат: "

mov eax,edi
call iprintLF    ; Печать результата (5)

mov eax,rem
call sprint      ; Печать "Остаток от деления: "

mov eax,edx
call iprintLF    ; Печать остатка (1)

call quit        ; Завершение программы
```

```
lab6-3.asm [----] 13 L:[ 1+10 11/ 41] *([*][X]
;-----
; Программа вычисления выражения f(x) = (4 * 6 + 2) / 5
;-----

%include 'in_out.asm' ; подключение внешнего файла

SECTION .data
div: DB 'Результат: ',0
rem: DB 'Остаток от деления: ',0

SECTION .text
GLOBAL _start

_start:
; ---- Вычисление выражения ----
mov eax,4 ; EAX = 4
mov ebx,6 ; EBX = 6
mul ebx ; EAX = 4 * 6 = 24

add eax,2 ; EAX = 24 + 2 = 26

xor edx,edx ; Обнуляем EDX перед делением
1Пощь 2Соан 3Блок 4За~на 5Копия 6Пе~ть 7Поиск 8Уд~ть
```

Рисунок 2.16: Измененный текст программы из файла lab6-3.asm

Далее я создала исполняемый файл и проверила его работу (Рисунок 2.17).

```
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ./lab6-3
Результат: 5
Остаток от деления: 1
vsgrigoreva@fedora:~/work/arch--pc/lab06$
```

Рисунок 2.17: Запуск программы из файла lab6-3.asm

Затем необходимо было рассмотреть программу вычисления варианта задания по номеру студенческого билета. В данном случае число, над которым необходимо проводить арифметические операции, вводится с клавиатуры. Ввод с клавиатуры осуществляется в символьном виде и для корректной работы арифметических операций в NASM символы необходимо преобразовать в числа. Для этого может быть использована функция `atoi` из файла `in_out.asm`.

Я создала файл `variant.asm` в каталоге `~/work/arch-pc/lab06` (Рисунок 2.18).

```
vsgrigoreva@fedora:~/work/arch--pc/lab06$ touch ~/work/arch--pc/lab06/variant.asm
```

Рисунок 2.18: Создание файла `variant.asm`

Затем в этот файл я записала текст программы из листинга 5 в файл `variant.asm` (Рисунок 2.19).

```
;-----
; Программа вычисления варианта
;-----

%include 'in_out.asm'

SECTION .data
msg: DB 'Введите № студенческого билета: ',0
rem: DB 'Ваш вариант: ',0

SECTION .bss
```

```
x: RESB 80
```

```
SECTION .text
```

```
GLOBAL _start
```

```
_start:
```

```
mov eax, msg
```

```
call sprintf
```

```
mov ecx, x
```

```
mov edx, 80
```

```
call sread
```

```
mov eax, x ; вызов подпрограммы преобразования
```

```
call atoi ; ASCII кода в число, `eax=x`
```

```
xor edx, edx
```

```
mov ebx, 20
```

```
div ebx
```

```
inc edx
```

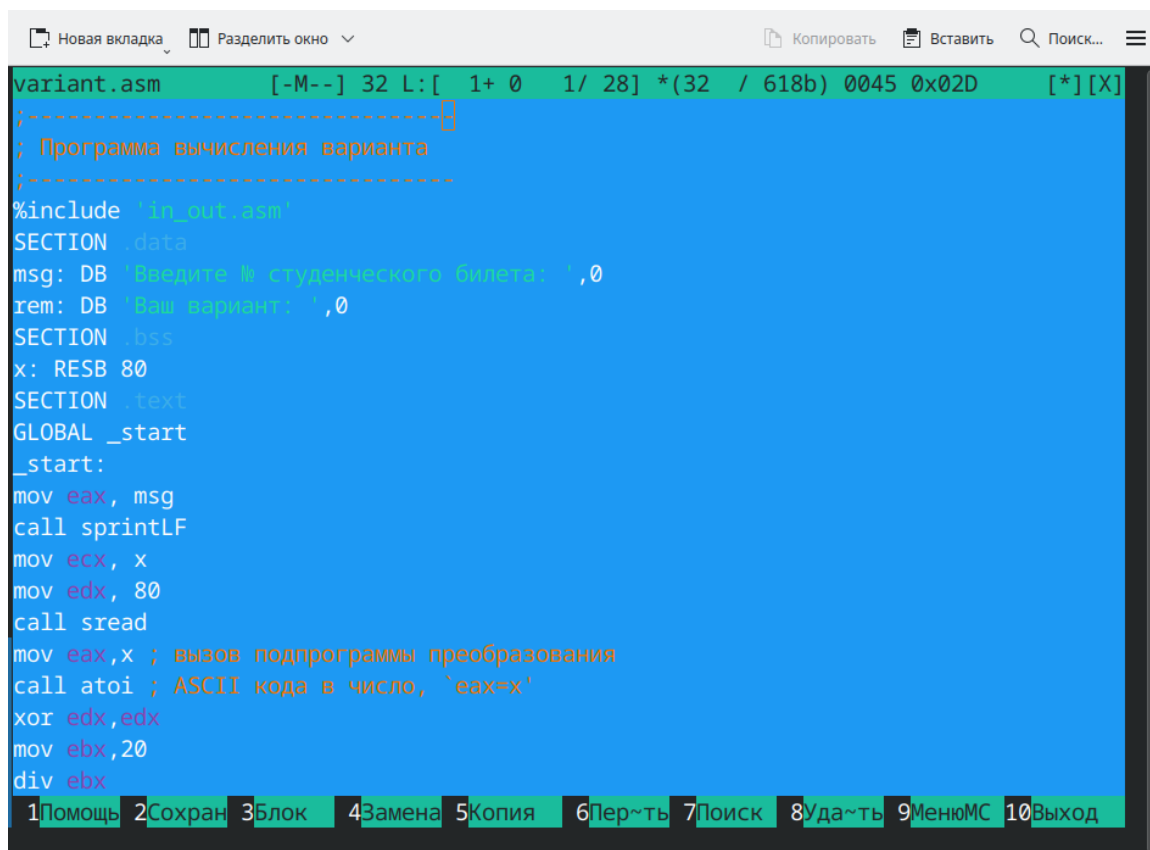
```
mov eax, rem
```

```
call sprintf
```

```
mov eax, edx
```

```
call iprintLF
```

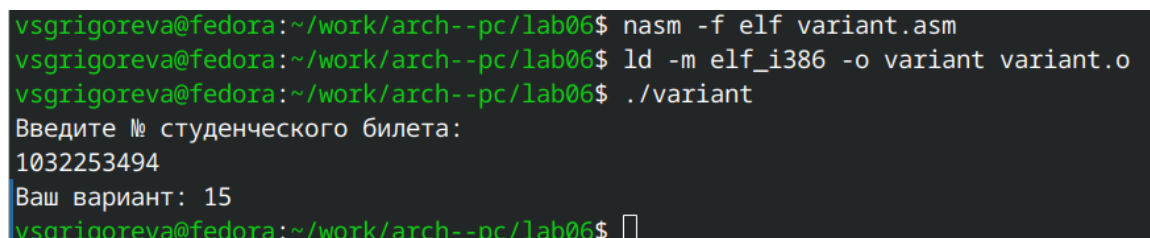
```
call quit
```



```
variant.asm [-M--] 32 L:[ 1+ 0 1/ 28] *(32 / 618b) 0045 0x02D [*] [X]
;-----
; Программа вычисления варианта
;-----
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите № студенческого билета: ',0
rem: DB 'Ваш вариант: ',0
SECTION .bss
x: RESB 80
SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprintf
mov ecx, x
mov edx, 80
call sread
mov eax, x ; вызов подпрограммы преобразования
call atoi ; ASCII кода в число, 'eax=x'
xor edx, edx
mov ebx, 20
div ebx
1Помощь 2Сохран 3Блок 4Замена 5Копия 6Пер~ть 7Поиск 8Уда~ть 9МенюМС 10Выход
```

Рисунок 2.19: Текст программы из файла variant.asm

Затем создала исполняемый файл и запустила его (Рисунок 2.20). У меня 15 вариант.



```
vsgrigoreva@fedora:~/work/arch--pc/lab06$ nasm -f elf variant.asm
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ld -m elf_i386 -o variant variant.o
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ./variant
Введите № студенческого билета:
1032253494
Ваш вариант: 15
vsgrigoreva@fedora:~/work/arch--pc/lab06$
```

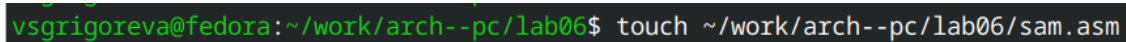
Рисунок 2.20: Запуск программы из файла variant.asm

Ответы на вопросы: 1. Какие строки листинга 6.4 отвечают за вывод на экран сообщения „Ваш вариант:“? Ответ: `mov eax,rem` и `call sprintf` 2. Для чего используются следующие инструкции? `mov ecx, x` `mov edx, 80` `call sread` Ответ: для ввода номера студенческого билета с клавиатуры и сохранения введенных символов в переменной

х 3. Для чего используется инструкция «call atoi»? Ответ: для преобразования символов в числа. 4. Какие строки листинга 6.4 отвечают за вычисления варианта? Ответ: `xor edx, edx mov ebx, 20 div ebx inc edx` 5. В какой регистр записывается остаток от деления при выполнении инструкции «div ebx»? Ответ: в регистр EDX 6. Для чего используется инструкция «inc edx»? Ответ: для увеличения значения регистра EDX на 1 7. Какие строки листинга 6.4 отвечают за вывод на экран результата вычислений? Ответ: `mov eax, rem call sprint mov eax, edx call iprintLF`

3 Задание для самостоятельной работы

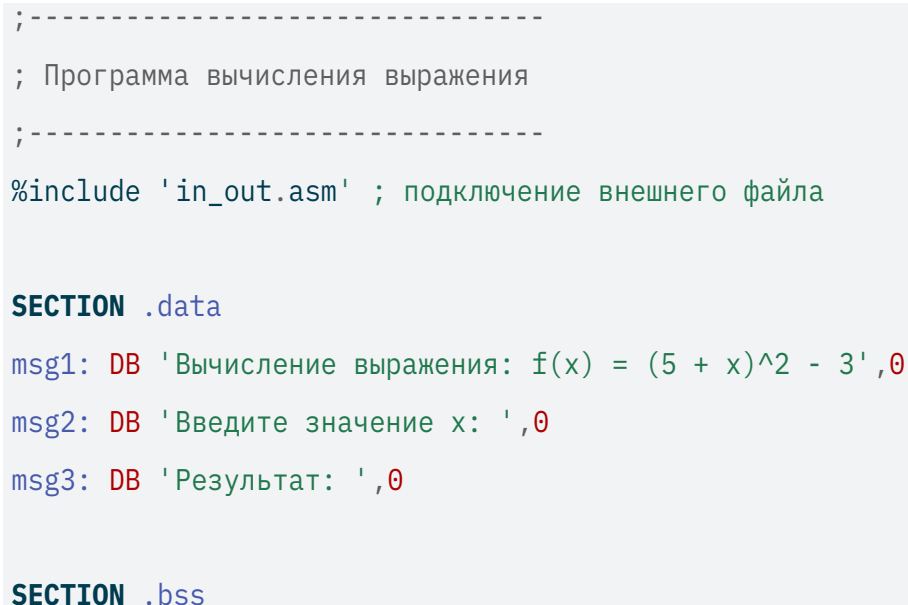
Сначала я создала файл sam.asm (Рисунок 3.1).



```
vsgrigoreva@fedora:~/work/arch--pc/lab06$ touch ~/work/arch--pc/lab06/sam.asm
```

Рисунок 3.1: Создание файла sam.asm

Затем я написала программу вычисления выражения $y = f(x) = (5 + x)^2 - 3$. Программа должна была выводить выражение для вычисления, выводить запрос на ввод значения x_1 , вычислять заданное выражение в зависимости от введенного x_1 , выводить результат вычислений (Рисунок 3.2). Текст программы приведен в листинге 6.



```
;-----  
; Программа вычисления выражения  
;-----  
%include 'in_out.asm' ; подключение внешнего файла  
  
SECTION .data  
msg1: DB 'Вычисление выражения: f(x) = (5 + x)^2 - 3',0  
msg2: DB 'Введите значение x: ',0  
msg3: DB 'Результат: ',0  
  
SECTION .bss
```



```
x: RESB 80 ; буфер для ввода
```

```
SECTION .text
```

```
GLOBAL _start
```

```
_start:
```

```
    ; ---- Вывод выражения ----
```

```
    mov eax, msg1
```

```
    call sprintf
```

```
    ; ---- Ввод x ----
```

```
    mov eax, msg2
```

```
    call sprintf
```

```
    mov ecx, x
```

```
    mov edx, 80
```

```
    call sread
```

```
    mov eax, x
```

```
    call atoi          ; преобразование строки в число, результат в EAX
```

```
    ; ---- Вычисление  $f(x) = (5 + x)^2 - 3$  ----
```

```
    add eax, 5          ; EAX = 5 + x
```

```
    mov ebx, eax        ; EBX = (5 + x)
```

```
    mul ebx             ; EAX = (5 + x)^2
```

```
    sub eax, 3          ; EAX = (5 + x)^2 - 3
```

```
    mov edi, eax        ; сохранить результат
```

```
    ; ---- Вывод результата ----
```

```

mov eax, msg3
call sprint
mov eax, edi
call iprintLF

; ---- Завершение ----
call quit

```

```

sam.asm [-M--] 13 L: [ 1+ 0 1/ 46] *(13 /1226b) 0045 0x02D [*][X]
;-----
; Программа вычисления выражения
;-----
#include 'in_out.asm' ; подключение внешнего файла

SECTION .data
msg1: DB 'Вычисление выражения: f(x) = (5 + x)^2 - 3',0
msg2: DB 'Введите значение x: ',0
msg3: DB 'Результат: ',0

SECTION .bss
x: RESB 80 ; буфер для ввода

SECTION .text
GLOBAL _start

_start:
; ---- Вывод выражения ----
mov eax, msg1
call sprintLF

; ---- Ввод x ----

```

Рисунок 3.2: Текст файла sam.asm

Далее я создала исполняемый файл и проверила его работу для значений 5 и 1 (Рисунок 3.3). Программа корректно посчитала и завершила работу.

```
vsgrigoreva@fedora:~$ cd ~/work/arch--pc/lab06vsgrigoreva@fedora:~/work/arch--pc/lab06$ nasm -f
elf sam.asm
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ld -m elf_i386 -o sam sam.o
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ./sam
Вычисление выражения:  $f(x) = (5 + x)^2 - 3$ 
Введите значение x: 5
Результат: 97
vsgrigoreva@fedora:~/work/arch--pc/lab06$ ./sam
Вычисление выражения:  $f(x) = (5 + x)^2 - 3$ 
Введите значение x: 1
Результат: 33
vsgrigoreva@fedora:~/work/arch--pc/lab06$
```

Рисунок 3.3: Запуск файла sam.asm

4 Вывод

В ходе выполнения работы я освоила арифметические инструкции языка ассемблера NASM.